# 6.2. Data Encoding

**Previous Section:**

6.1. Data Normalization

**Contents:**



Encoding (also known as **feature encoding**) is another essential data transformation technique in data preprocessing. While normalization prepares **numerical** features by scaling their values, encoding converts **categorical** (text) features into numerical representations that machine learning algorithms can understand.

For example, humans can easily distinguish between majors such as **“AI”**, **“IT”**, and **“CSC”**, or English proficiency levels such as **“A1”**, **“A2”**, **“B1”**, and **“B2”**. However, machine learning models only process numbers, making encoding a necessary step before model training.

For this section, we will recreate a simplified student dataset containing both numerical and categorical features.

```
import numpy as np
import pandas as pd
```

```
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

n = len(names)

df = pd.DataFrame({
    "Name": names,
    "Age": np.random.randint(18, 21, n),
    "Gender": np.random.choice(["Male", "Female"], n),
    "Major": np.random.choice(["AI", "IT", "CSC", "SW"],
n),
    "English Level": np.random.choice(["A1", "A2", "B1", "B
2"], n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head())
```

Although Pandas provides the convenient `pd.get_dummies()` function for One-Hot Encoding, many other encoding techniques can be implemented easily using Pandas' powerful mapping and vectorized operations.

## 1. Label Encoding

Label Encoding assigns each category a unique integer value.

- For example, `Male -> 0; Female -> 1`

This is one of the simplest encoding techniques and can be implemented using `map()`.

```
gender_map = {"Male": 0, "Female": 1}
df["Gender_Label"] = df["Gender"].map(gender_map)

print(df[["Gender", "Gender_Label"]].head())
```

The same idea can be applied to any categorical feature.

```
major_map = {"AI": 0, "IT": 1, "CSC": 2, "SW": 3}
df["Major_Label"] = df["Major"].map(major_map)

# Remove the Gender and Major column
df = df.drop(columns=['Gender', 'Major'])

print(df[["Name", "Gender_Label", "Major_Label"]].head())
```

**Advantages**

- Very easy to implement.
- Requires only one numerical column.
- Memory efficient.

**Disadvantages**

The encoded numbers may unintentionally introduce an order that does not actually exist.

- For example, `AI -> 0; IT -> 1; CSC -> 2; SW -> 3` does **not** imply that **SW > CSC > IT > AI**, yet many machine learning algorithms may interpret it this way.

## 2. One-Hot Encoding

One-Hot Encoding creates a new binary column for every category. Instead of assigning a single integer, each category becomes its own feature.

For example:

| Major | Major_AI | Major_IT | Major_CSC | Major_SW |
|---|---|---|---|---|
| AI | 1 | 0 | 0 | 0 |

| Major | Major_AI | Major_IT | Major_CSC | Major_SW |
|---|---|---|---|---|
| CSC | 0 | 0 | 1 | 0 |
| IT | 0 | 1 | 0 | 0 |

- Pandas provides this functionality directly through `pd.get_dummies()`.

```
major_onehot = pd.get_dummies(df["Major"],
                              prefix="Major")
print(major_onehot.head())
```

- We can also encode multiple columns simultaneously.

```
df_onehot = pd.get_dummies(df, columns=["Gender", "Major"])
print(df_onehot.head())
```

- We can replace the True/False with 0/1 using replace(), which is very important for model training

```
df_onehot = df_onehot.replace({True: 1, False: 0})
print(df_onehot)
```

**Note:** After applying encoding techniques, it is often practical to remove the original categorical columns since their information is now represented by the newly created binary columns.
Fortunately, `pd.get_dummies()` for One-Hot Encoding does this automatically when the `columns` parameter is used, replacing the original columns with their encoded counterparts.

### Advantages

- No artificial ordering is introduced.
- Works well for most machine learning algorithms.
- The most widely used categorical encoding method.

**Disadvantages**

- Datasets with many unique categories may produce hundreds or even thousands of additional columns.
- Higher memory usage for high-cardinality features.

## 3. Ordinal Encoding

Ordinal Encoding is used when the categories have a **natural order**. Unlike Label Encoding, the numerical values intentionally represent ranking.

- For example, `A1 -> 1; A2 -> 2; B1 -> 3; B2 -> 4`

```
english_map = {"A1": 1, "A2": 2, "B1": 3, "B2": 4}

df["English_Ordinal"] = df["English Level"].map(english_ma
p)
print(df[["English Level", "English_Ordinal"]].head())
```

Although Ordinal Encoding appears similar to Label Encoding, its numerical values carry meaningful ordering.

Examples include: Education Level; Satisfaction Rating; Military Rank; Language Proficiency; Product Quality

## 4. Other Encoding Methods

Besides the techniques discussed above, several other encoding methods are commonly used in machine learning:

- **Binary Encoding** converts categories into integers, represents those integers in binary form, and splits the binary digits into multiple columns. It is efficient for features with many unique categories.
- **Target Encoding** replaces each category with the average value of the target variable associated with that category. It is frequently used in supervised learning but should be applied carefully to avoid data leakage.
- **Frequency (Count) Encoding** replaces each category with its occurrence count or frequency in the dataset. This is useful for high-cardinality

categorical features.

**Note: Multi-Hot Encoding**
Multi-Hot Encoding is used when a single sample belongs to **multiple categories simultaneously**.
For example, a student may participate in multiple clubs.

```
club_df = pd.DataFrame({
    "Name": ["James", "Chris", "Emma"],
    "Clubs": [
        ["AI Club", "Football"],
        ["Music", "Football"],
        ["AI Club", "Music"]
    ]
})

club_df
```

Each club becomes its own binary feature.

```
clubs = club_df["Clubs"].explode()

multi_hot = pd.crosstab(
    clubs.index,
    clubs
)

result = pd.concat(
    [club_df["Name"], multi_hot],
    axis=1
)

print(result)
```

**Advantages**

- Represents multiple labels naturally.

- Common in recommendation systems and multi-label classification.

**Disadvantages**

- Produces many columns when there are numerous possible labels.

## Summary

Encoding transforms categorical data into numerical values that machine learning algorithms can process.

- Pandas provides `pd.get_dummies()` for One-Hot Encoding, while other encoding methods can be implemented easily using `map()`, `crosstab()`, and other vectorized operations.
- **Label Encoding** assigns each category a unique integer but may unintentionally introduce an artificial ordering.
- **One-Hot Encoding** creates one binary column per category and is the most commonly used encoding technique because it avoids introducing false relationships.
- **Ordinal Encoding** is appropriate when categories have a meaningful ranking, such as education levels or language proficiency.
- **Multi-Hot Encoding** represents samples that belong to multiple categories simultaneously and is widely used in recommendation systems and multi-label classification.
- Other specialized encoding techniques include **Binary Encoding**, **Target Encoding**, and **Frequency (Count) Encoding**, each designed for specific types of datasets and machine learning tasks.

## References

https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html

https://www.geeksforgeeks.org/pandas/python-pandas-get_dummies-method/

https://www.geeksforgeeks.org/machine-learning/categorical-data-encoding-techniques-in-machine-learning/

## Next Section:

- 7.1. Data Merging & Joining